

Mercari 가격 제안 문제 선형회귀 모델링 보고서

LinearRegression · Ridge · Lasso 비교 및 RMSLE 기반 성능 분석

분석 대상 코드 : Mercari_Price_Suggestion_LinearRegression_pure.ipynb

분석 대상 데이터 : mercari_train.tsv (1,482,535행 × 8열)

작성일 : 2026-08-27

0. 개요

본 보고서는 Kaggle Mercari Price Suggestion Challenge 데이터를 대상으로, 상품의 이름·설명·브랜드·카테고리·상태·배송 정보로부터 판매 가격을 예측하는 선형 모델링 과정을 정리한다.

LinearRegression, Ridge, Lasso 세 모델을 동일한 피처 조합으로 비교했고, Ridge의 정규화 강도(alpha)는 수동 그리드 탐색과 Optuna 자동 탐색 두 방식으로 최적화했다.

최종적으로 Ridge(alpha=5)가 RMSLE 0.4792로 가장 낮은 오차를 기록했으며, 이는 실제 가격 대비 약 -38% ~ +61% 수준의 오차 범위에 해당한다. 세부 해석은 8장에서 다룬다.

1. 서론

1.1 연구 배경 및 필요성

Mercari는 판매자가 직접 상품 가격을 입력해야 하는 중고거래 플랫폼이다.

동일한 상품이라도 판매자마다 가격을 다르게 책정하면 거래 성사율과 플랫폼 신뢰도가 떨어진다. 상품 정보만으로 적정 가격대를 자동으로 제안할 수 있다면 판매자의 의사결정 부담을 줄일 수 있다.

1.2 과제 목적

- 상품명(name), 상품 설명(item_description), 브랜드, 카테고리, 상품 상태, 배송비 부담 여부로 가격을 예측한다.
- LinearRegression / Ridge / Lasso 세 가지 선형 회귀 모델의 성능을 RMSLE 기준으로 비교한다.
- Ridge의 alpha를 수동 그리드와 Optuna로 각각 탐색해 최적 정규화 강도를 찾는다.

2. 데이터셋 소개

2.1 데이터 개요

표 2-1. mercari_train.tsv 기본 정보

항목	내용
행 수	1,482,535
열 수	8
구분자	탭(wt)

표 2-2. 컬럼 구성

컬럼명	설명	결측치 수
train_id	고유 식별자	0
name	상품명	0
item_condition_id	상품 상태(1~5, 숫자가 클수록 상태가 나쁨)	0
category_name	카테고리 (대/중/소 계층, '/'로 구분)	6,327
brand_name	브랜드명	632,682
price	판매 가격 (목표 변수)	0
shipping	배송비 부담 여부(0/1)	0
item_description	상품 설명 텍스트	6

brand_name의 결측 비율은 약 42.7%로 가장 크다. 이는 4장의 브랜드 보완 작업의 근거가 된다.

3. 타깃 변수(Price) 분포 및 로그 변환

원본 price 분포는 저가 상품에 몰려 있고 고가 쪽으로 긴 꼬리를 가지는 형태다(오른쪽 꼬리 분포).

이런 비대칭 분포를 그대로 학습하면 소수의 고가 상품이 오차(손실)를 과도하게 지배해 모델이 저가 상품 예측에 소홀해질 수 있다.

이에 `np.log1p(price)` 로 로그 변환을 적용했다. 변환 후 분포는 좌우 대칭에 가까운 종 모양으로 바뀌었다(노트북 히스토그램 비교 셀 기준).

로그 변환의 두 번째 근거: 평가지표인 RMSLE 자체가 로그 오차 기반 지표다. 학습 목표를 로그 스케일로 맞추면 손실 함수와 평가지표의 방향이 일치해 최적화가 더 직접적으로 이뤄진다.

4. 피처 탐색 및 전처리

4.1 배송비·상품 상태 분포

표 4-1. shipping 값 분포

값	의미	건수	비율
0	구매자가 배송비 부담	819,435	55.3%
1	판매자가 배송비 부담	663,100	44.7%

표 4-2. item_condition_id 값 분포

값	건수	비율
1 (최상)	640,549	43.2%
2	375,479	25.3%
3	432,161	29.2%
4	31,962	2.2%
5 (최하)	2,384	0.2%

4.2 카테고리 계층 분할(대/중/소)

category_name을 '/' 기준으로 분리해 대분류(cat_dae), 중분류(cat_jung), 소분류(cat_so) 세 컬럼을 새로 만들었다. 결측인 경우 세 컬럼 모두 'Other_Null'로 채웠다.

표 4-3. 대분류(cat_dae) 분포

대분류	건수
Women	664,385
Beauty	207,828
Kids	171,689
Electronics	122,690
Men	93,680
Home	67,871
Vintage & Collectibles	46,530
Other	45,351
Handmade	30,842
Sports & Outdoors	25,342
Other_Null(결측)	6,327

중분류는 114종, 소분류는 871종이다. Women 카테고리가 전체의 약 45%를 차지해 카테고리 분포가 한쪽으로 크게 치우쳐 있다.

소분류(cat_so, 871종)는 최종 모델 피처에서 제외했다. 근거는 두 가지다.

- ① 대분류·중분류와 계층적으로 정보가 겹친다.
- ② 종류가 많아 각 범주별 표본 수가 적고, 선형모델의 회귀 계수 추정이 불안정해질 위험이 있다.

4.3 브랜드 결측치 보완

brand_name 결측 632,682건 중, 상품명(name)에 결측이 아닌 다른 행의 브랜드명이 그대로 포함된 경우를 찾아 역채움했다.

방식: 결측이 아닌 brand_name 값의 집합을 만든 뒤, 상품명 단어를 3단어·2단어·1단어 조합 순으로 확인해 이 집합에 속하는 조합이 있으면 그 값으로 채운다.

표 4-4. 브랜드 결측 보완 결과

구분	건수
보완 전 결측	632,682
상품명에서 역추론으로 채운 건수	148,714
역추론 후에도 남은 결측(Other_Null 처리)	483,968

이 방식은 새로운 브랜드 값을 만들지 않고 기존 브랜드 목록 안에서만 채우므로, 이후 브랜드 원-핫 인코딩의 차원 수에는 영향을 주지 않는다.

4.4 결측치 최종 처리

brand_name, category_name, item_description의 남은 결측치는 모두 'Other_Null' 문자열로 채웠다. 처리 후 전 컬럼의 결측치는 0건이다.

5. 피처 엔지니어링(벡터화)

5.1 텍스트 피처 벡터화

표 5-1. name / item_description 벡터화 설정

컬럼	방식	설정	결과 차원
name	CountVectorizer	min_df=2, max_features=20,000	1,482,535 × 20,000
item_description	TfidfVectorizer	max_features=20,000, ngram_range=(1,3), stop_words='english'	1,482,535 × 20,000

name은 평균 길이가 짧아 특정 단어가 한 상품명 안에서 반복되는 경우가 드물다. 따라서 단어 출현 빈도(TF) 방식인 CountVectorizer로 충분하다고 판단했다.

item_description은 문장이 길고, 여러 상품에서 공통으로 쓰이는 단어(설명 문구)가 많다. 이런 경우 TF-IDF로 특정 상품 설명에서만 두드러지는 단어에 가중치를 주는 것이 유리하다.

min_df=2는 1건에서만 등장하는 오탈자·희귀어를 걸러 노이즈를 줄이기 위한 설정이다. max_features=20,000은 차원을 벡터화 단계에서 직접 제한해 이후 SVD 등 추가 축소 없이

연산량을 통제하기 위한 설정이다.

5.2 범주형 피처 원-핫 인코딩

brand_name, item_condition_id, shipping, cat_dae, cat_jung 다섯 개 컬럼을 `LabelBinarizer(sparse_output=True)` 로 원-핫 인코딩했다.

희소 행렬(sparse matrix)로 인코딩한 이유는, brand_name처럼 범주 종류가 많은 컬럼을 밀집 배열(dense array)로 만들면 대부분이 0인 값에 메모리를 낭비하기 때문이다.

5.3 전체 피처 결합

위 벡터화·인코딩 결과를 `scipy.sparse.hstack` 으로 결합한 전체 피처 행렬의 크기는 아래와 같다.

표 5-2. 전체 피처 행렬 크기

행(샘플 수)	열(피처 수)
1,482,535	44,941

참고(산출값) : name(20,000) + item_description(20,000) + item_condition_id(5) + shipping(1, 이진 클래스라 LabelBinarizer가 1열 반환) + cat_dae(11) + cat_jung(114) = 40,131. 전체 44,941에서 이를 뺀 4,810이 brand_name 원-핫 차원(고유 브랜드 값 개수)으로 역산된다. 이 값은 노트북에 직접 출력되지 않아 다른 셀의 출력값으로부터 계산한 추정치임을 밝힌다.

6. 평가지표: RMSLE

RMSLE(Root Mean Squared Logarithmic Error)는 다음과 같이 정의한다.

$$\text{RMSLE} = \sqrt{(1/n) \sum (\log(1+\hat{y}_i) - \log(1+y_i))^2}$$

모델은 로그 변환된 price(3장)로 학습했으므로, 예측값과 실제값을 `np.exp`로 원래 가격 스케일로 복원한 뒤 RMSLE를 계산했다(`evaluate_org_price` 함수).

RMSLE는 절대 오차가 아닌 상대(비율) 오차 성격의 지표다. 즉 \$10짜리 상품의 \$5 오차와 \$100짜리 상품의 \$50 오차를 비슷한 비중으로 취급한다. 가격처럼 저가~고가 폭이 넓은 데이터에 적합한 이유다.

7. 모델링 및 결과

7.1 공통 학습·평가 방식

전체 피처 행렬을 `train_test_split(test_size=0.2, random_state=42)` 로 8:2 분할해 학습/평가했다. 교차검증(k-fold)은 적용하지 않은 단일 홀드아웃 검증이다.

7.2 LinearRegression: item_description 포함 여부

표 7-1. LinearRegression 결과

조건	RMSLE
item_description 포함(전체 피쳐)	0.4818
item_description 제외	0.5154

item_description을 포함하면 RMSLE가 0.5154 → 0.4818로 약 0.0335 감소한다. 상품 설명 텍스트가 가격 예측에 유의미한 정보를 담고 있음을 뜻한다.

7.3 Ridge(L2 정규화)

표 7-2. Ridge alpha별 RMSLE (전체 피쳐, solver='lsqr')

alpha	0.01	0.1	1	5	10
RMSLE	0.4812	0.4811	0.4800	0.4792	0.4798

alpha를 높이면 회귀 계수가 0 쪽으로 축소되어 분산은 줄지만 편향이 커진다(과소적합 방향). alpha를 낮추면 LinearRegression에 가까워져 분산이 커진다(과대적합 방향).

실험 범위 안에서는 alpha=5에서 RMSLE 0.4792로 가장 낮았고, alpha=10에서는 다시 소폭 상승했다. 즉 alpha 5 부근이 편향-분산 균형점에 가깝다.

7.4 Lasso(L1 정규화)

표 7-3. Lasso alpha별 RMSLE (전체 피쳐, max_iter=3000)

alpha	0.01	0.1	1	5	10
RMSLE	0.6924	0.7511	0.7511	0.7511	0.7511

Lasso는 모든 alpha에서 Ridge보다 크게 낮은 성능을 보였다. 특히 alpha=0.1 이상에서는 RMSLE가 완전히 동일한 값(0.7511)으로 고정된다.

alpha=0.1 이상에서 결과가 동일하다는 것은, 이 구간부터 대부분의 회귀 계수가 정확히 0으로 수렴해 모델이 사실상 절편(평균 예측) 수준으로 단순화되었을 가능성을 시사한다.

원인 추정: L1 정규화는 상관된 더미 변수(원-핫 인코딩된 브랜드·카테고리, TF-IDF 단어 조합)들 중 하나만 남기고 나머지를 0으로 만드는 경향이 Ridge(L2)보다 강하다. 44,941차원의 희소 피쳐 조합에서는 이 효과가 과도하게 작용해 정보 손실이 커진 것으로 판단된다.

7.5 Optuna 기반 Ridge alpha 탐색

수동 그리드 탐색에서 alpha=5 부근이 최적으로 나타나, 이 부근을 포함하도록 alpha 탐색 범위를 $1e-3 \sim 1e2$ (로그 스케일)로 설정하고 Optuna로 15회 시행했다.

표 7-4. Optuna 탐색 결과 요약

항목	값
시행 횟수	15
탐색 범위	$\alpha \in [0.001, 100]$, log-uniform
최적 α	2.3785
최적 RMSLE	0.47942

Optuna가 찾은 최적값($\alpha=2.3785$, RMSLE 0.47942)은 앞선 수동 그리드의 $\alpha=5$ 결과 (RMSLE 0.47918)보다 근소하게 높다. 차이는 0.00024로 실질적으로 미미한 수준이다.

두 결과를 종합하면, α 1~10 구간에서 RMSLE 곡선이 상당히 평탄(flat)하며, 이 구간 어디를 택해도 성능 차이는 크지 않다고 해석할 수 있다.

7.6 전체 결과 비교

표 7-5. 전 모델 RMSLE 종합 비교

모델	조건	RMSLE
LinearRegression	item_description 포함	0.4818
LinearRegression	item_description 제외	0.5154
Ridge	$\alpha=0.01$	0.4812
Ridge	$\alpha=0.1$	0.4811
Ridge	$\alpha=1$	0.4800
Ridge	$\alpha=5$ (최적)	0.4792
Ridge	$\alpha=10$	0.4798
Ridge(Optuna)	$\alpha=2.3785$	0.4794
Lasso	$\alpha=0.01$	0.6924
Lasso	$\alpha \geq 0.1$	0.7511

8. 결과 해석: RMSLE의 실제 가격 의미

RMSLE는 로그 오차 지표라, 예측 가격이 실제 가격 대비 대략 어느 비율 범위에 있는지로 환산해 해석할 수 있다.

근사식: $\text{예측 가격} \approx \text{실제 가격} \times e^{(\pm \text{RMSLE})}$

표 8-1. 모델별 RMSLE를 가격 범위로 환산 (근사치)

모델(조건)	RMSLE	실제가 \$10일 때	실제가 \$50일 때	실제가 \$100일 때
Ridge alpha=5 (최적)	0.4792	\$6.19 ~ \$16.15	\$30.97 ~ \$80.74	\$61.93 ~ \$161.48
LinearRegression(설명 포함)	0.4818	\$6.18 ~ \$16.19	\$30.89 ~ \$80.95	\$61.77 ~ \$161.90
LinearRegression(설명 제외)	0.5154	\$5.97 ~ \$16.74	\$29.87 ~ \$83.71	\$59.73 ~ \$167.42
Lasso(alpha≥0.1, 사실상 절편모델)	0.7511	\$4.72 ~ \$21.19	\$23.59 ~ \$105.97	\$47.18 ~ \$211.93

주의: 위 범위는 RMSLE를 로그오차의 대표적 크기로 보고 역산한 근사치이며, 실제 개별 예측이 이 구간 안에 들어간다는 통계적 보장은 아니다. RMSLE는 오차 제곱의 평균을 다시 제곱근한 값이라 일부 극단적으로 빗나간 예측(긴 꼬리)의 영향을 받으며, 실제 오차 분포는 이 근사 구간보다 더 넓게 퍼져 있을 수 있다.

그럼에도 모델 간 상대 비교는 유효하다. 최적 Ridge는 Lasso(절편모델 수준) 대비 오차 범위 폭이 뚜렷하게 좁다. 예를 들어 \$50 상품 기준 Ridge는 약 \$50 폭(≈\$31~\$81)인 반면 Lasso는 약 \$82 폭(≈\$24~\$106)으로 두 배 이상 넓다.

9. 종합 평가

- 텍스트 피처, 특히 item_description은 가격 예측력에 실질적으로 기여한다(7.2절).
- 세 모델 중에서는 Ridge가 가장 낮은 RMSLE를 기록해 가장 안정적이었다.
- Lasso는 고차원 희소 피처(44,941차원) 환경에서 계수를 과도하게 0으로 축소해, 이번 데이터·피처 조합에는 적합하지 않았다.
- Optuna 자동 탐색은 수동 그리드 탐색과 유사한 최적 지점(alpha 2~5 구간)을 확인해, 최적점의 위치가 신뢰할 만함을 뒷받침한다.

10. 결론

본 분석에서 최종적으로 채택할 모델은 **Ridge(alpha=5)**이며, RMSLE 0.4792를 기록했다.

이는 실제 가격 대비 대략 -38% ~ +61% 오차 범위(8장 근사 해석 기준)에 해당한다. 절대적인 정확도로 보면 여전히 오차 폭이 크므로, 실무에 바로 적용하기보다는 가격 "구간 제안"의 참고 지표로 활용하는 편이 적절하다.

11. 한계 및 향후 과제

- **검증 방식:** 단일 홀드아웃(8:2) 검증만 수행했다. k-fold 교차검증을 적용하면 RMSLE 추정치의 변동성을 함께 확인할 수 있다.
- **미사용 피처:** 소분류(cat_so, 871종)는 카디널리티 문제로 제외했다. 임베딩 또는 빈도 기반 인코딩 등 다른 처리 방식을 시도해볼 여지가 있다.
- **테스트 제출 미실시:** 본 노트북은 mercari_train.tsv 내부 홀드아웃 평가까지만 수행했고, Kaggle 실제 테스트셋 제출 결과는 포함하지 않는다.
- **모델 계열의 한계:** 선형모델은 피처 간 비선형 상호작용을 반영하지 못한다. 트리 기반 모델 (LightGBM 등)과의 비교는 이번 분석 범위에 포함되지 않았으며, 후속 과제로 제안한다.
- **Lasso 결과의 추가 검증:** $\alpha \geq 0.1$ 구간에서 계수가 실제로 0에 수렴했는지는 계수 배열을 직접 확인하지 않아 정황적 추정이다. 후속 분석에서 `model.coef_`의 0 비율을 직접 확인하면 더 확실하다.

12. 참고 자료

- 분석 노트북: Mercari_Price_Suggestion_LinearRegression_pure.ipynb
- 데이터: mercari_train.tsv (Kaggle Mercari Price Suggestion Challenge)

부록. 노트북 내 정의 함수

표 A-1. 노트북에서 정의한 주요 함수

함수명	역할
split_cat	category_name을 '/' 기준 대/중/소분류로 분리. 결측 시 'Other_Null' 3개 반환
find_brand_in_name	상품명 단어 조합(3→2→1단어)을 기존 브랜드 목록과 대조해 브랜드명 역추론
rmsle	실제값·예측값(원가격 스케일)으로 RMSLE 계산
evaluate_org_price	로그 스케일 예측값·실제값을 expm1로 복원한 뒤 rmsle 호출
model_train_predict	피처 리스트 결합(hstack) → 8:2 분할 → 학습 → 예측까지 수행하는 공통 파이프라인